Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

D. Anti-pattern Catalog 2
D.1. Kiến trúc & thiết kế 2
D.2. Thiết kế API 3
D.3. Giao tiếp 3
D.4. Giao dịch & dữ liệu 4
D.5. Bảo mật & Gateway 5
D.6. Chuyển đổi & observability 6
D.7. Triển khai & DevOps 7

PHỤ LỤC D

Anti-pattern Catalog

Tổng hợp 43 anti-pattern từ các mục sai lầm thường gặp, lỗ hổng, cạm bẫy và rủi ro của 12 chương, nhóm theo 7 chủ đề. Khi gặp một vấn đề trong hệ thống, tra bảng theo chủ đề, xem mức độ và cách phòng tránh, rồi đọc chương tương ứng để hiểu ngữ cảnh; ưu tiên các mục mức nghiêm trọng, nhất là mục liên quan đến mất dữ liệu và bảo mật.

Bốn mức độ nghiêm trọng:

- **NGHIÊM TRỌNG (critical, 8 mục)**: dữ liệu mất hoặc hệ thống sập.
- **CAO (high, 22 mục)**: ảnh hưởng nghiêm trọng đến tính đúng đắn hoặc khả năng vận hành.
- **TRUNG BÌNH (medium, 12 mục)**: tăng độ phức tạp hoặc nợ kỹ thuật (technical debt) đáng kể.
- **THẤP (low, 1 mục)**: bất tiện, dễ khắc phục.

D.1. Kiến trúc & thiết kế

Bảng D.1 tóm tắt các anti-pattern về kiến trúc và thiết kế, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.1: *Anti-pattern kiến trúc và thiết kế*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
1	**Hype-Driven Microservices** — Lựa chọn kiến trúc microservices chạy theo trào lưu, không xuất phát từ bài toán thực tế của tổ chức	TRUNG BÌNH	Làm tăng độ phức tạp của hệ thống mà không mang lại lợi ích thực tiễn	Xuất phát từ vấn đề (Khung ra quyết định) thay vì áp đặt giải pháp từ đầu	1
2	**Big Bang Migration** — Chuyển đổi tổng thể một cách đột ngột từ kiến trúc nguyên khối (monolith) sang microservices, bỏ qua bước trung gian	CAO	Tạo ra Distributed Monolith — mang nhược điểm của cả hai mô hình	Tái cấu trúc (refactor) ranh giới các module rõ ràng *trước* khi tiến hành tách dịch vụ	1

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
3	**Premature Service Decomposition** — Phân rã dịch vụ trước khi tổ chức lại nhân sự (ví dụ: một đội ngũ phải ôm đồm 10 dịch vụ và triển khai chúng cùng lúc)	TRUNG BÌNH	Tăng gánh nặng vận hành (overhead) mà không đạt được sự tự chủ (autonomy) thực sự	Tổ chức lại đội ngũ nhân sự *cùng lúc* hoặc *trước* khi tách dịch vụ (định luật Conway)	1
4	**Entity-driven Design** — Thiết kế theo hướng thực thể dữ liệu (1 thực thể = 1 bảng, 1 dịch vụ = 1 nhóm bảng từ sơ đồ ERD)	CAO	Ranh giới dịch vụ bị phụ thuộc vào lược đồ dữ liệu (schema) thay vì miền nghiệp vụ (domain)	Thiết kế xuất phát từ hành vi và sự kiện nghiệp vụ (sử dụng Event Storming), không bắt đầu từ dữ liệu	2
5	**Nano-services** — Tách mỗi thực thể thành một dịch vụ siêu nhỏ mặc dù chúng luôn thay đổi cùng nhau	CAO	Độ trễ tăng cao, gây khó khăn lớn cho việc theo dõi và gỡ lỗi (debugging)	Phân rã theo Bounded Context, không phân rã theo từng thực thể	2
6	**Shared Library Abuse** — Lạm dụng thư viện dùng chung (shared library) vì sự tiện lợi, trộn lẫn logic nghiệp vụ với các nghiệp vụ cắt ngang	TRUNG BÌNH	Tạo ra sự phụ thuộc ngầm khi triển khai; thay đổi thư viện ảnh hưởng toàn bộ hệ thống	Phân định rõ ràng: nghiệp vụ cắt ngang (cross-cutting concern) thì dùng chung, logic nghiệp vụ thì tách biệt	2

D.2. Thiết kế API

Bảng D.2 tóm tắt các anti-pattern về thiết kế API, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.2: *Anti-pattern thiết kế API*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
7	**Exposing Entities via API** — Trả trực tiếp thực thể dữ liệu (entity) qua API mà không qua đối tượng truyền dữ liệu (DTO), làm lộ cấu trúc nội bộ	CAO	Gây ra breaking change khi thay đổi cơ sở dữ liệu và tiềm ẩn rủi ro bảo mật	Luôn dùng DTO để tách biệt mô hình nội bộ (internal model) khỏi hợp đồng giao tiếp (external contract)	3
8	**Inconsistent Naming Convention** — Quy tắc đặt tên (naming convention) không đồng nhất, trộn lẫn số ít/số nhiều và kiểu viết (kebab-case/camelCase)	THẤP	Việc sửa đổi quy tắc đặt tên về sau sẽ gây ra breaking change trên diện rộng cho các consumer	Thống nhất một quy tắc chung và bắt buộc tuân thủ ngay từ những dòng mã đầu tiên	3
9	**Missing Standard Error Format** — Bỏ qua định dạng lỗi chuẩn hóa; mỗi dịch vụ trả về một cấu trúc báo lỗi riêng biệt	TRUNG BÌNH	Consumer phải xử lý quá nhiều định dạng, gây khó khăn cho việc gỡ lỗi	Dùng một bộ xử lý ngoại lệ tập trung, trả về định dạng lỗi thống nhất cho mọi dịch vụ	3

D.3. Giao tiếp

Bảng D.3 tóm tắt các anti-pattern về giao tiếp, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.3: *Anti-pattern giao tiếp*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
10	**Synchronous Service Chain** — Thiết lập chuỗi gọi đồng bộ (A → B → C → D), mỗi dịch vụ phải chờ đợi dịch vụ tiếp theo phản hồi	CAO	Độ trễ cộng dồn, làm giảm tính khả dụng (ví dụ: xin chữ ký 4 phòng ban, một người đi vắng là tắc nghẽn toàn bộ)	Chuyển đổi sang giao tiếp bất đồng bộ (async) cho các luồng không trọng yếu, áp dụng API Composition	4

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
11	**Missing Circuit Breaker** — Thiếu cơ chế ngắt mạch (Circuit Breaker), tiếp tục gọi đến dịch vụ đang gặp sự cố mà không dừng lại	NGHIÊM TRỌNG	Gây ra hiệu ứng sụp đổ dây chuyền (cascading failure), làm tê liệt toàn bộ hệ thống	Áp dụng thư viện chịu lỗi (ví dụ Resilience4j) kết hợp ngắt mạch, thử lại (retry) và giới hạn thời gian (timeout)	4
12	**Retry Without Backoff** — Thử lại ngay lập tức khi gặp lỗi mà không có độ trễ tăng dần (exponential backoff)	CAO	Dịch vụ đang quá tải bị dồn thêm áp lực, nhanh chóng cạn kiệt tài nguyên và sụp đổ	Áp dụng độ trễ tăng dần kèm độ lệch ngẫu nhiên (exponential backoff + jitter): 1s → 2s → 4s	4
13	**Premature Auto-commit** — Tự động xác nhận (auto-commit) đã nhận thông điệp trước khi tiến trình xử lý hoàn tất	NGHIÊM TRỌNG	Thông điệp bị mất vĩnh viễn nếu consumer gặp sự cố giữa chừng	Xác nhận thủ công (manual offset commit) — chỉ đánh dấu hoàn tất *sau khi* đã xử lý thành công	5
14	**Missing Dead Letter Topic** — Thiếu Dead Letter Topic (DLT) để chứa thông điệp lỗi, dẫn đến việc thử lại vô hạn hoặc gây tắc nghẽn luồng xử lý	CAO	Thông điệp lỗi (Poison Pill) chặn đứng toàn bộ các thông điệp xếp sau nó	Định tuyến các thông điệp lỗi sang một DLT riêng sau số lần thử lại nhất định	5
15	**Non-idempotent Consumer** — Consumer không bảo đảm idempotency, mặc định giả định mỗi thông điệp chỉ đến một lần	CAO	Thông điệp bị lặp (do hệ thống thử lại) dẫn đến sai lệch dữ liệu	Chống trùng nguyên tử theo eventId: unique constraint hoặc CAS, ghi dấu và cập nhật nghiệp vụ trong cùng transaction	5
16	**Familiarity-driven Broker Selection** — Lựa chọn message broker dựa trên thói quen thay vì đặc thù bài toán (ví dụ: dùng queue kiểu tiêu thụ rồi xóa cho bài toán cần phát lại sự kiện, chỉ vì đội ngũ đã quen thuộc)	TRUNG BÌNH	Thiếu tính năng cần dùng (ví dụ phát lại luồng sự kiện), buộc chuyển đổi tốn kém	Đánh giá dựa trên nhu cầu thực tế: lưu vết bền vững (Kafka) hay định tuyến thông minh (RabbitMQ)	5

D.4. Giao dịch & dữ liệu

Bảng D.4 tóm tắt các anti-pattern về giao dịch và dữ liệu, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.4: *Anti-pattern giao dịch và dữ liệu*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
17	**Two-Phase Commit (2PC) in Microservices** — Áp dụng giao thức 2PC để cố gắng duy trì giao dịch phân tán (distributed transaction) xuyên suốt các dịch vụ	CAO	Gây tắc nghẽn (blocking), tạo điểm lỗi tập trung (SPOF) và làm giảm tính khả dụng	Chấp nhận tính nhất quán cuối cùng (eventual consistency) và dùng Saga Pattern	6
18	**Missing Compensating Action** — Bỏ qua việc định nghĩa các hành động bù trừ (compensation), chỉ tập trung vào kịch bản thành công	NGHIÊM TRỌNG	Dữ liệu không nhất quán, các bản ghi bị kẹt trạng thái vĩnh viễn	Mỗi bước có thể bù trừ (compensatable) của Saga phải có hành động bù trừ (compensating action) đã kiểm thử; xác định rõ giao dịch bản lề (pivot) và các bước có thể thử lại (retriable), không bắt mọi bước phải có bù trừ	6

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
19	**Saga Without Timeout** — Triển khai Saga nhưng thiếu cơ chế giới hạn thời gian (timeout) để xác định thời điểm giao dịch thất bại	CAO	Các yêu cầu xử lý bị kẹt ở trạng thái đang xử lý (`PENDING`) vô thời hạn	Thiết lập tiến trình nền (scheduled job) để kiểm tra quá hạn và tự động kích hoạt bù trừ	6
20	**Missing Semantic Lock** — Không sử dụng khóa ngữ nghĩa (Semantic Lock), cho phép can thiệp vào dữ liệu đang nằm trong một chuỗi Saga	CAO	Gây ra xung đột tương tranh (race condition), đọc dữ liệu bẩn (dirty read) và kết quả sai lệch	Thiết lập trạng thái khóa ngay khi Saga bắt đầu và chỉ gỡ khóa khi chuỗi giao dịch hoàn tất	6
21	**Premature Database Decomposition** — Phân tách cơ sở dữ liệu quá sớm khi chưa nắm rõ các mô hình truy cập dữ liệu (access pattern)	TRUNG BÌNH	Dẫn đến nhu cầu kết nối dữ liệu (JOIN) liên tục, buộc phải xây dựng các luồng sự kiện phức tạp	Bắt đầu bằng việc phân tách lược đồ (schema) trong cùng hệ quản trị, quan sát trước khi tách DB vật lý	7
22	**Overusing CQRS** — Lạm dụng kiến trúc CQRS cho mọi bài toán, kể cả các tác vụ thêm/đọc/sửa/xóa (CRUD) cơ bản	TRUNG BÌNH	Làm tăng độ phức tạp hệ thống lên gấp nhiều lần cho những dữ liệu ít khi thay đổi	Chỉ áp dụng CQRS khi luồng đọc thực sự phức tạp, lưu lượng lớn, hoặc cần kết hợp nhiều dịch vụ	7
23	**Data Duplication Without Source of Truth** — Nhân bản dữ liệu mà không xác định rõ nguồn sự thật duy nhất (Source of Truth)	CAO	Gây xung đột khi nhiều dịch vụ cùng chỉnh sửa; không thể xác định phiên bản dữ liệu nào là chuẩn xác	Mỗi thực thể dữ liệu chỉ được phép có một chủ sở hữu duy nhất — các bản sao đều phải ở dạng chỉ đọc	7
24	**Ignoring Replication Lag** — Bỏ qua độ trễ nhân bản dữ liệu (replication lag): vừa ghi xong chuyển sang đọc ngay lập tức dẫn đến thấy dữ liệu cũ	TRUNG BÌNH	Gây bối rối cho người dùng (ví dụ: sinh viên nộp bài thành công nhưng bảng điểm chưa cập nhật)	Tùy bài toán: "Cập nhật lạc quan" (Optimistic UI) cho luồng đơn giản; với dữ liệu phụ thuộc nhiều nguồn (leaderboard), dùng synchronous read model / read-your-writes	7

D.5. Bảo mật & Gateway

Bảng D.5 tóm tắt các anti-pattern về bảo mật và Gateway, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.5: *Anti-pattern bảo mật và Gateway*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
25	**Bespoke JWT Validation** — Mỗi Gateway/service tự chọn thư viện, thuật toán và quy tắc issuer/audience khác nhau	CAO	Quy tắc xác minh trôi lệch giữa các nơi: nơi chấp nhận, nơi từ chối; bộ xác minh lỏng có thể chấp nhận token không dành cho mình	Chuẩn hóa bộ xác minh, JWKS và hợp đồng xác minh; Gateway kiểm tra ở biên (ingress), resource service xác minh bằng chứng có chữ ký rồi tự phân quyền cục bộ	8–9
26	**Missing Rate Limiting** — Bỏ qua cơ chế giới hạn lưu lượng (Rate Limiting) tại API Gateway	NGHIÊM TRỌNG	Dễ bị tấn công từ chối dịch vụ hoặc quá tải khi lưu lượng tăng đột biến (ví dụ: hàng nghìn sinh viên cùng lúc tải lại trang đăng ký tín chỉ khiến dịch vụ ngừng phản hồi)	Áp dụng cơ chế giới hạn lưu lượng tại Gateway (thuật toán Token Bucket với Redis)	8

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
27	**Overambitious API Gateway** — Nhồi nhét quá nhiều logic nghiệp vụ và logic biến đổi dữ liệu vào API Gateway	CAO	Gateway biến thành điểm nghẽn (bottleneck); mỗi lần thay đổi đều phải triển khai lại toàn bộ	Gateway chỉ nên đảm nhận các nghiệp vụ cắt ngang: định tuyến, xác minh token ở biên, giới hạn lưu lượng và ghi log	8
28	**Long-lived JWT** — Cấp phát token JWT không có thời hạn hoặc thời hạn quá dài (ví dụ: access token có hiệu lực 30 ngày)	NGHIÊM TRỌNG	Nếu token bị đánh cắp, tin tặc sẽ nắm giữ quyền truy cập hệ thống trong thời gian rất dài	Sử dụng access token có vòng đời ngắn (15–60 phút) kết hợp cùng refresh token để cấp lại	9
29	**Storing JWT in Local Storage** — Lưu trữ token JWT tại kho chứa cục bộ (localStorage) của trình duyệt trên máy khách	NGHIÊM TRỌNG	Nếu có XSS, mã độc JavaScript đọc và đánh cắp được token	HttpOnly cookie kèm Secure, SameSite và chống CSRF (§9.6); vẫn phải phòng XSS	9
30	**Hard-coded Roles** — Gắn cố định (hard-code) các vai trò trực tiếp vào mã nguồn (ví dụ: `if role == "ADMIN"`) rải rác khắp nơi	CAO	Mỗi khi bổ sung vai trò mới, lập trình viên phải sửa đổi mã nguồn ở nhiều nơi, dễ xảy ra sai sót	Dùng RBAC, đặt ánh xạ vai trò sang quyền hạn (permission) ở một chỗ, rồi kiểm tra theo quyền hạn thay vì so sánh tên vai trò rải rác trong mã	9

D.6. Chuyển đổi & observability

Bảng D.6 tóm tắt các anti-pattern về chuyển đổi và observability, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.6: *Anti-pattern chuyển đổi và observability*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
31	**Distributed Monolith** — Phân rã dịch vụ nhưng vẫn dùng chung cơ sở dữ liệu, thư viện và phải triển khai đồng loạt	NGHIÊM TRỌNG	Gánh chịu toàn bộ độ phức tạp của microservices nhưng không thu được lợi ích (ví dụ: các khoa độc lập nhưng dùng chung một máy in)	Áp dụng cơ sở dữ liệu riêng cho từng dịch vụ và chỉ dùng chung các thư viện tối giản	10
32	**Big Bang Rewrite** — Đập bỏ xây mới: viết lại toàn bộ hệ thống từ đầu thay vì chuyển đổi dần dần từng phần	CAO	Tốn kém nguồn lực bảo trì cả hai hệ thống song song; thời điểm chuyển giao (switch day) liên tục bị trì hoãn	Áp dụng mẫu thiết kế Strangler Fig — bóc tách từng phần nhỏ và đưa vào sử dụng ngay	10
33	**Rushed Migration** — Chuyển đổi quá vội vã (ví dụ: tách 20 dịch vụ trong 2 tháng chỉ để thỏa mãn "quyết định kiến trúc")	CAO	Xác định sai ranh giới hệ thống, dẫn đến phải thiết kế, xây dựng lại và tiêu tốn gấp đôi công sức	Chuyển đổi từng dịch vụ một, bắt đầu từ những cấu phần mang lại giá trị nghiệp vụ cao nhất	10
34	**Lift-and-shift Without Redesign** — Chia các phần của monolith thành nhiều đơn vị triển khai qua mạng mà vẫn giữ coupling, không thiết kế lại (container hóa nguyên khối thì vẫn là bước chuyển tiếp hợp lệ)	CAO	Phát sinh độ trễ mạng và các rủi ro vận hành trong khi hệ thống vẫn bị phụ thuộc chặt chẽ vào nhau	Thiết kế lại mô hình nghiệp vụ cho từng dịch vụ và sử dụng Anti-Corruption Layer (lớp dịch mô hình)	10
35	**Infrastructure Over-engineering** — Phức tạp hóa hạ tầng quá mức cần thiết (ví dụ: dựng Kubernetes + Istio phức tạp chỉ cho 3 dịch vụ nhỏ)	TRUNG BÌNH	Đội ngũ tiêu tốn phần lớn thời gian để bảo trì hạ tầng thay vì tập trung phát triển các tính năng cốt lõi	Khởi đầu đơn giản với Docker Compose khi một máy đáp ứng được tải, và chỉ mở rộng quy mô hạ tầng khi thực sự cần	10

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
36	**Error-only Logging** — Chỉ ghi nhận nhật ký (log) khi có lỗi xảy ra, bỏ qua việc ghi nhận các luồng yêu cầu thông thường	CAO	Khi sự cố xảy ra, kỹ sư không có đủ dữ liệu bối cảnh (context) để tiến hành gỡ lỗi	Ghi nhật ký có cấu trúc (structured logging) cho mọi yêu cầu kèm các trường liên kết (`requestId`, `traceId`, `correlationId`)	11
37	**Metric Overload or Starvation** — Thu thập số liệu (metric) quá dư thừa (gây nhiễu) hoặc quá nghèo nàn (chỉ đo CPU/RAM)	TRUNG BÌNH	Dư thừa gây ra tình trạng nhiễu loạn cảnh báo. Nghèo nàn tạo ra các điểm mù trong hệ thống	Tập trung vào các chỉ số trọng tâm (SLI/SLO) như: độ trễ, tỷ lệ lỗi (error rate), thông lượng (throughput)	11
38	**Alert Fatigue** — Cảnh báo tràn lan, mọi exception đều kích hoạt thông báo gửi qua email hoặc Slack	TRUNG BÌNH	Gây ra tình trạng mệt mỏi vì cảnh báo (alert fatigue), khiến đội ngũ bỏ lỡ các sự cố nghiêm trọng thực sự	Chỉ cấu hình cảnh báo dựa trên các *triệu chứng* ảnh hưởng trực tiếp đến người dùng (vi phạm SLO)	11
39	**Broken Correlation ID** — Đứt gãy chuỗi định danh (Correlation ID), dữ liệu truy vết không được truyền qua các luồng bất đồng bộ	CAO	Bức tranh truy vết phân tán bị đứt đoạn, khiến việc theo dõi yêu cầu xuyên suốt hệ thống trở nên bất khả thi	Truyền `traceparent` qua Kafka header, `correlationId` trong event envelope, và đưa cả hai vào MDC khi xử lý	11

D.7. Triển khai & DevOps

Bảng D.7 tóm tắt các anti-pattern về triển khai và DevOps, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.7: *Anti-pattern triển khai và DevOps*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
40	**Friday Deployment** — Thực hiện triển khai (deploy) vào lúc đội ngũ không sẵn sàng ứng cứu (ví dụ: cập nhật tính năng chiều thứ Sáu / cuối tuần)	CAO	Nếu xảy ra sự cố sập hệ thống (downtime) khi không có nhân sự trực, hậu quả sẽ kéo dài đến đầu tuần	Nên ưu tiên triển khai vào các buổi sáng giữa tuần, khi toàn bộ đội ngũ đang ở trạng thái sẵn sàng xử lý sự cố	12
41	**Kubernetes for Trivial Workloads** — Áp dụng Kubernetes cho các hệ thống quy mô nhỏ, đơn giản với tư duy "Các công ty lớn dùng thì chúng ta cũng nên dùng"	TRUNG BÌNH	Gây ra gánh nặng vận hành rất lớn cho các đội ngũ quy mô nhỏ (2–3 thành viên)	Sử dụng Docker Compose khi một máy đáp ứng được tải. Chỉ dùng Kubernetes khi *thực sự cần* phân tán đa máy chủ	12
42	**Environment-specific Image Builds** — Đóng gói các bản dựng (image) khác nhau cho từng môi trường riêng biệt (dev/staging/prod)	CAO	Gây ra lỗi môi trường (chạy tốt trên staging nhưng lỗi trên production) do sự sai khác về bản dựng	Xây dựng một lần, triển khai mọi nơi (Build once, deploy everywhere) — tách biệt cấu hình qua biến môi trường	12
43	**Missing Rollback Plan** — Thiếu phương án hoàn tác (rollback plan): khi phiên bản mới gặp lỗi nghiêm trọng, không có quy trình khôi phục rõ ràng	NGHIÊM TRỌNG	Gây hoảng loạn, dẫn đến việc chỉnh sửa trực tiếp trên production và sinh ra thêm lỗi mới	Mọi đợt triển khai bắt buộc phải có quy trình hoàn tác (rollback procedure) được ghi chép rõ ràng	12